Review paper on Fine Tuning LLM

Kashish Pimpalshende **Sanju Mandal** **Gayatri Chippawar**

Abstract: A thorough analysis of fine-tuning techniques for Transformer-based Large Language Models (LLMs) is presented in this review paper, with an emphasis on both small scale and large-scale architectures. We start by summarizing the development of Transformer models, from contemporary decoder-only generative model like Llama and Mistral to encoder-only framework like BERT. The main fine-tuning methods, and alignment techniques like Direct Preference Optimization (DPO) and Reinforcement Learning from Human Feedback (RLHF), are methodically reviewed in this paper. We examine the fine-tuning of large models like Llama 3.8B and Mistral 7B, as well as compact model like TnyLlmana, Gemma-2B, and Phi-3 Mini, through in-depth case studies, emphasizing their domain adaptation capabilities, resource requirements, and performance trade-offs. We also go over evaluation metrics that are frequently used to evaluate efficiency, accuracy, and alignment, and we highlight some of the main issues, such ad computational cost, hallucination, and the limitations of smaller models. All things considered, this review of a methodical summary of existing fine-tuning techniques, useful results, and potential future paths for tailoring LLMs to task-specific, effective, and aligned real-world applications.

1. Introduction

Recurrent models like LSTM and GRU dominated sequence tasks but suffered from sequential computation that limited parallelization, especially for long sequences. Although attention mechanism improved dependency modelling, they were mostly tied to recurrent architectures. The Transformer removed recurrence entirely and relied solely on self-attention to model global dependencies. This enabled massive parallelization, faster training, and state-of-the-art performance in sequence transduction tasks [1].

BERT (Bidirectional Encoder Representations from Transformers, 2018) pioneered encoder-only models with masked language modelling and next sentence prediction [2]. It demonstrated the power of pretraining and fine tuning, achieving state-of-the-art result on benchmarks like GLUE and SQuAD. However, Bert is not generative. Subsequent research shifted to decoder-only architectures for generative

capabilities, leading to large language models (LLMs) like GPT, Llama, and Mistral [2], [3].

Fine tuning remains essential to adapt both small and large models for domain specific tasks, instruction following and efficient deployment [3]. This review presents: an overview of transformer architecture, fine tuning methods case studies of small and large LLMs, evaluation metrics, challenges and future direction

2. Transformer Architecture Overview

Transformers revolutionized deep learning by replacing recurrent architectures with self-attention mechanisms, enabling parallel processing and long-rang dependency modelling. Introduced in the paper **"Attention IS All You Need"**. The transformer architecture consists of stacked encoder and decoder blocks facilitated by multi-head attention, positional encodings and residual connections [1].

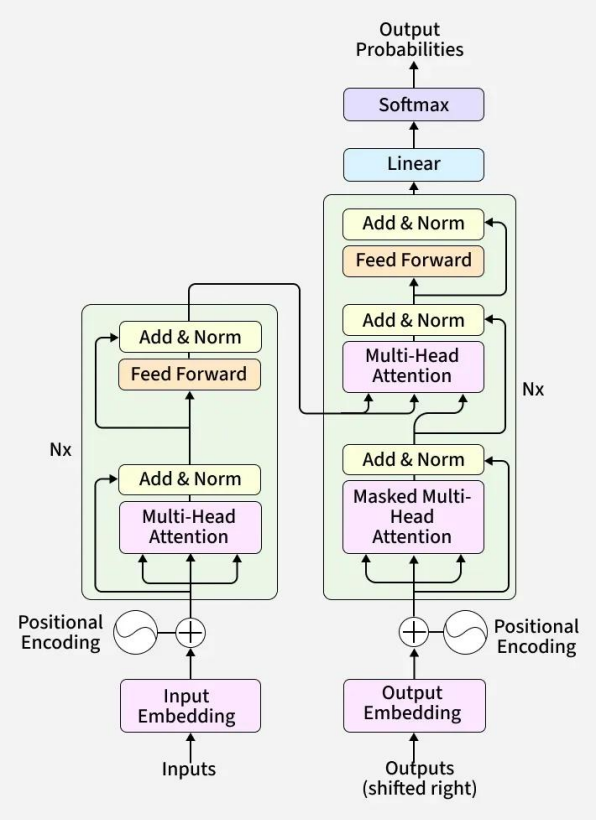

Figure 1. Transformer architecture

2.1 Core components of Transformers

a) Multi-Head Self-Attention

Self-attention allows every token to interact with every other token in a sequence. Multiple heads different types of relationships- syntax, semantics and long-range

dependencies-simultaneously. This is how model learn context beyond simple word prediction.

b) Positional Encoding

Transformer do not read tokens sequentially, positional encoding injects order information using sinusoidal functions. This enables the model to understand "What come before and after", even without recurrence.

c) Feed-Forward Networks

Each Transformer block contains a feed forward network that applies non-linear transformations to each token independently. While attention extracts relationships, feed-forward network help define those features into meaningful representations.

d) Residual Connection and Layer Normalization

Deep networks often suffer from vanishing gradients. Residual connection allows gradient flow through layers, while layer normalization ensures stable training, making it possible to train model with billons of parameters.

2.2 Variants of Transformers Architectures

The Transformer variants have evolved through three primary Transformer configurations: Encoder-Only, Decoder-Only and Encoder-Decoder architectures, these frameworks utilizes the Transformer's core components in unique way to handle different linguistic challenges.

.

a) Encoder Only Model

Encoder-only model utilizes a multi-layer bidirectional Transformer encoder to create deep language representations by simultaneously analyzing context from both directions in every layer. By employing a Masked Language Model objective, it moves beyond unidirectional constraints to fully integrate surrounding text into its token representations[4].

A standout development in this field is Devlin et al.'(2018) BERT (Bidirectional Encoder Representations from Transformers) model. BERT introduced two key pre-training goals:

- Masked Language Modeling (MLM):

$$L_{MLM} = -\sum_{i \in M} \log P(x_i \mid x_{\setminus M})$$

  Where tokens are hidden to force the model to learn deep bidirectional context.

- Next Sentence Prediction (NSP): a binary classification task used to teach the model how to understand the relationship between two sentences.

b) Decoder Only Model

Decoder-Only architecture utilizes transformer decoder with casual(left-to-right) self-attention, enabling token-by-token generation where the system predicts each subsequent symbol by conditioning exclusively on the preceding context[7].

Training objective is based on Causal Language Modelling (CLM):

$$Loss_{CLM} = -\sum_{t=1}^{T} \log P(x_t \,|x_{<t})$$

Bidirectional encoder such as BERT focuses on deeply integrating context from both sides, whereas unidirectional decoders specialize in generative tasks by predicting the next token based exclusively on preceding text [4].

As language models grow in parameter count and are exposed to extensive, high-quality datasets, they undergo a qualitative shift where complex skills like multi-step reasoning and few-shot learning appear spontaneously [5].

Beyond just increasing model size, techniques like instruction tuning and reinforcement Learning from Human Feedback (RLHF) are used to align system outputs with user intent. This process refines high-capacity models like GPT, Llama and Mistral to better follow complex instructions and act as helpful assistants rather than just predicting the next token [6], [8].

c) Encoder Decoder Model

Encoder–Decoder architectures retain the dual-stack design of the original Transformer, where the encoder converts inputs into representations and the decoder uses a cross-attention mechanism to generates outputs conditioned on those representations [1]. These models are primarily utilized in conditional generation tasks, such as translation and summarization [10].

Notable Models: T5, BART, mBART.

Strength: Highly effective for input-to-output tasks (e.g., translation, paraphrasing).

Limitation: Less flexible than decoder-only models for open-ended reasoning and dialogue [9].

3. Fine Tuning Approches :

Fine-tuning is a critical stage in adapting pre-trained language models to specific downstream tasks. Depending on computational resources, task requirements, and model size, different strategies are employed. The following are the most widely adopted fine-tuning paradigms in large language model research:

3.1 Supervised Fine Tuning

Supervised fine tuning involves training a pre-trained model on task -specified labelled datasets using traditional supervised learning. The objective is to optimize the model to produce correct outputs for given inputs.

Key Feature: task-specific alignment using gold-standard labels

Common Use: classification, summarization, instruction-following [11].

3.2 Full Fine Tuning

Full fine-tuning updates all model parameters, allowing complete task adaptation. Although highly effective, it is computationally expensive and memory-intensive, especially for large-scale models exceeding billions of parameters.
Key Advantage: Maximum Model flexibility and performance
Limitation: Require significant GPU resources [11].

3.3 Parameter-Efficient Fine Tuning (PEFT)

PEFT technique aim to reduce computational cost by fine-tuning only a small subset of parameters or by introducing lightweight task-specific modules such as LoRA, Adapters, or Prefix Tuning.
Key Feature: Suitable for large models with limited compute.
Mechanism: Keeps original weights frozen, train only additional modules [11].

3.4 Reinforcement Learning with Human Feedback (RLHF)

RLHF refines model alignment by utilizing a reward mechanism grounded in human preference rankings rather than just static labels. This approach moves beyond simple pattern matching to specifically optimize for qualitative traits like helpfulness, safety and factual accuracy.
Use case: GPT-3.5, GPT-4 [11].

4. Fine- Tuning Small Models

4.1 TinyLlama

a) Introduction to TInyLlama:

TinyLlama 1.1b highlights the growing emphasis on efficiency in today's model designs. With just 1.1 billion parameters, it is built to run fast and use very little memory, which makes it well-suited for a wide range of devices , including systems with limited computing power. Because of this, TinyLlama lowers the barrier using strong language models in places where large-scale models simply aren't practical. This kind of accessibility is seen as an important step toward making advanced AI development and deployment available to more people [12].

b) Fine Tuning TinyLlama for domain specific tasks:

The constraints imposed by resource-limited pretraining often become evident when smaller models are evaluated on tasks that require domain-specific knowledge that was insufficiently represented in their training data. For instance, the initial TinyLlamaChat model showed weak performance on basic mathematical aptitude questions, a shortcoming that can be directly attributed to the lack of dedicated mathematics-focused datasets during its pretraining stage [12].

This failure pattern highlights a unique role for fine-tuning in SLMs: it functions as a mechanism for foundational knowledge injection rather than mere

performance refinement[12]. Due to its compact parameter size, TinyLlama is an exceptional candidate for efficient domain fine-tuning. By adapting the model on a specialized dataset, such as a targeted mathematics corpus, researchers can rapidly improve the quality of generation in that domain, moving the model from a state of functional failure to robust utility[12]. For models of this scale, highly targeted fine-tuning compensates for resource limitation during base model training by structurally remediating specific, critical knowledge gaps, allowing researchers with limited hardware to customize an SLM for their specific needs[12].

Table 1. TinyLlama 1.1B vs. Domain fine tuning:

Metric	Baseline	Fine tuned
Domain Specific Accuracy (Math Aptitude)	**Extremely Poor**	**Quality of Generation improves significantly**
Training Resource Constraint	**High**	**Low**
Primary Limitation Addressed	**Pretraining data deficit**	**-**

4.2 Gemma-2B

a) Introduction to Gemma-2B

Gemma is a family of lightweight, state-of-the-art open models developed by Google DeepMind, leveraging the research and technology used to create the Gemini models. These models are designed to provide strong performance across various academic benchmarks related to language understanding, reasoning and safety while being openly accessible to research and development.

Gemma 2B model is build on Transformer decoder-based architecture, featuring 18 layers, model dimension of 2048 and context window of 8192 tokens. To ensure high computational efficiency at a small scale this model contain several modern architectural features. These include Multi-Query Attention (MQA) that performs multi-query attention at smaller scale, Rotary Positional Embeddings (RoPE) are used in each layer instead of absolute positional embeddings, GeGLU activation function replaced the standard ReLU , RMSNorm is used to stabilize the training process [13].

b) Multi-Stage Instruction Tuning: SFT and RLHF for Helpfulness and Safety

Achieving robust instruction following capabilities and safety alignment in SLMs requires sophisticated post-training techniques, often mirroring the complex pipelines employed for frontier models. Gemma 2B utilizes a multi-stage process involving Supervised Fine-Tuning (SFT) followed by Reinforcement Learning from Human Feedback (RLHF)[13].

Gemma 2B models are fine-tuned using SFT on a diverse mix of text-only, English-only synthetic, human-generated prompt-response pairs and Data mixtures for SFT are selected based on LM-based side-by-side evaluations [13]. The SFT data undergoes several filtering processes to remove sensitive information, unsafe and toxic model output, mistaken self-identification data, duplicated examples [13]. Filtering also includes subsets of data that encourage Hedging and attribution to minimize the hallucinations[13]. After supervised fine- tuned model then undergoes further fine tuning using RLHF, where this RLHF process involves reward model training where the model is trained on human preference data, and policy optimization using a novel RL algorithm [13]. Gemma's multi-stage instruction tuning represents a mature methodology for developing capable and safe language model[13]. This approach provides not just a technical recipe but a philosophical framework recognizing the responsibility of creating powerful technologies and responding with appropriate rigor and ethical commitment

Table 2. Gemma 2B vs Instruction tuned model

Metric	**Instruction tuning method**	**MMLU(5- shot, top-1)**
Baseline	**None**	**42.3**
Instruction tuned	**SFT+ RLHF(multi-stage alignment)**	**64.3**

4.3 Phi-3 Mini (3.8B)

a) Introduction to Phi-3 Mini:

Phi-3 Mini is a highly capable small language model developed by Microsoft that run locally on mobile devices while delivering performance comparable to much large models like Mixtral 8x7B and GPT-3.5. The model represents a paradigm shift in AI development, prioritizing data quality over model size through innovative data curation and synthetic data generation techniques [14]. The Phi-3 mini model isbuilds on transformer decoder architecture with 3.8 billion parameters, train on 3.3 trillion tokens, featuring 32 layers, context length of 4k tokens and extended to 128k tokens, hidden dimensions of 3072[14].

b) Fine tuning in Phi-3:

The process of alignment for Phi-3 Mini builds upon its strong pretraining foundation using supervised fine-tuning and Direct Preference Optimization (DPO). These post-training techniques are crucial for guaranteeing robust instruction-following behavior, adhering to safety guidelines, and ensuring optimal performance in a chat format [14].

In supervised Fine-Tuning model trains on curated instruction-response pairs, covering diverse domains to build board capabilities and establishes foundational instruction-following behavior [14]. The selection of DPO is important, as it is simpler and more reliable way to align a model using human preferences. Instead of first training a separate reward model like in traditional RLHF, DPO adjusts the model directly based on human preferences [14]. Since the model has already been trained on massive and well organize dataset of 3.3 trillion tokens, it already understands language and patterns extremely well. This strong foundation makes the fine-tuning stage easier and more effective, helping polishing model's behavior and stabilize its performance at a very high level [14].

Table 3. Phi-3 mini base vs Instruction-aligned checkpoint

Metric	Alignment strategy	MMLU score	MT-bench score
Baseline	**Base pretrained only**	**68.8%**	**8.38**
Phi-3 Mini-4K-instruct	**SFT+DPO**	**69%**	**8.38**

5 Fine- Tuning Large Models

5.1 Llama 3 8B instruct

a) Introduction to Llama:

The Llama 3 B Instruct model is a strong foundational model known for its solid performance on complex tasks, especially in instruction-following and technical analysis. Domain-adapted versions of this model, such as Foundation-Sec-8B-Instruct, have shown particularly in cyber threat intelligence applications [15].

b) Fine Tuning:

Fine-tuning Llama 3 highlights the strong impact of supplementing training data with carefully curated synthetic examples, particularly for specialized tasks such as classification and information extraction. One effective use case is adapting the model to extract MITRE ATT&CK techniques from unstructured cyber-threat reports. Internal evaluations showed that the security-focused version of the model, known as Foundation-Sec, surpassed the general-purpose Llama 3.1-8B model by more than % on

this classification task. These results underscore the considerable benefits of targeted security-domain pretraining followed by focused fine-tuning [15] .

Supervised Fine-Tuning (SFT) is mainly used to strengthen a model's core instruction-following abilities and is applied to Foundation-Sec-8B (pretrained cybersecurity base model), using standard post-training practices common in language model development [15]. Following SFT, Direct Preference Optimization (DPO) is used to further enhance instruction adherence and better align the model's outputs with human preferences [15]. Compared to more complex multi-stage reinforcement learning approaches such as Proximal Policy Optimization (PPO), DPO is considered simpler and is categorized as reinforcement learning techniques applied after SFT stage [15].

Table 4. Llama 3.1-8B-instruct vs Foundation-sec-8B-instruct

Model variant	MMLU	BBH	GSM8k	Alpaca Eval 2	MATH	IFEval	HumanEval
Llama 3.1-8B-Instruct	0.679	0.725	0.835	24.477	0.425	0.791	0.864
Foundation-sec-8B-instruct	0.602	0.677	0.817	35.453	0.411	0.811	0.843

5.2 Mistral 7B

a) Introduction to Mistral:

The Mistral 7B model is highly regarded for its performance within its complexity class, offering strong capabilities in language comprehension and chatbot-oriented tasks. As a widely adopted open-source model, it serves as a robust platform for researchers seeking to adapt a generalist LLM to specialized industry use cases without relying on proprietary systems [17].

b) Fine tuning:

A notable example of Mistral &B domain-specific fine-tuning is its adaptation for adaptive machine translation, particularly for Spanish-to-English translation in the medical field [17]. The goal of this work was to strengthen the model's in-context learning ability, allowing it to modify its translations on the fly when given zero-shot prompts (source text only) or one-shot prompts (source text plus a fuzzy-match example) [17].

The fine-tuning process uses QLoRA (Low-Rank Adaptation with 4-bit quantization), a Parameter-Efficient Fine-Tuning (PEFT) technique that allows for training with limited computational resources, such as a single NVIDIA A100 GPU [17]. The training data consisted of relatively small mixed dataset of 20,000 segments, evenly divided between zero-shot and one-shot prompt [17]. Notably, a single training epoch was sufficient to achieve the best result [17]. This adaptation substantially improved Mistral's ability to leverage in-context examples during translation tasks [17]. After fine tuning, Mistral 7B's

zero-shot translation quality matched that of specialized systems such as NLLB 3.3B, while its one-shot performance exceeded NLLB 3.3B and approached that of commercial large language models like ChatGPT (gpt-3.5-turbo) [17]. These findings demonstrate that effective, high-quality domain adaptation for mid-sized models can be achieved efficiently through PEFT methods and small, carefully curated datasets, producing outputs comparable to those of much large or task-specific systems that require far greater computational resources [17].

Table 5. Mistral 7B baseline vs. Fine tuned

Model	context	BLEU	chrF++	COMET
Mistral 7B (base)	Source only (zero-short)	42.88	66.03	69.56
Mistral 7B (base)	+Fuzzy (one shot)	47.71	69.25	76.37
Mistral 7B (fine tuned)	Source only (zero-short)	46.71	69.55	77.44
Mistral 7B (fine tuned)	+Fuzzy (one shot)	49.69	70.89	79.62

6 Evaluation Metrics

The evaluation framework for large language model are organized into three primary metric categories [18] :

6.1 Multiple-Classification (MC) Metrics

The evaluation is based on standard performance measures such as accuracy, recall, precision, F1 score, Micro-F1, Macro-F1, AUC/PRAUC and This Metrics are used to evaluate how accurately an LLM classifies text into two pr multiple labels using benchmark datasets with known single labels, similar to diagnostic-style evaluation [18].

6.2 Token-Similarity (TS) Metrics

The evaluation relies on the metrics such as Perplexity, BLUE, ROUGE, METEOR, and BertScore. This metrics are used to evaluate how closely LLM-generated text matches reference outputs, particularly in task like machine translation and text summarization where producing the correct sequence of words matters [18].

6.3 Question-Answering (QA) Metrics

The evaluation is based on measure such as strict accuracy, lenient accuracy, and reciprocal rank. This metrics are used to evaluate how effectively an LLM identifies

answer within a given context passage, focusing on predicting the correct start and end position of an answer as a constrained double-task problem [18].

7 Challenges

7.1 Limitation of Small Models

Although the efficiency of smaller LLMs, such as TinyLlama, make them desirable, their absolute capability to massive model is intrinsically limited [19].

7.2 High Computation power Requirement

The significant computational and memory requirenments for both training and high-throughput inference make LLM deployment difficult even with the use of efficient fine-tuning techniques (PEFT). Fine-tuning usually requires dedicated GPU memory of at least 16 GB of VRAM, even for compact models, which presents a significant hardware barrier for individual developers and small organizations [19].

7.3 Hallucination of model

The tendency of LLMs to produce outputs that are coherent but factually false or nonsensical is referred to as hallucination. For complex tasks, fine-tuning models may still produce irrelevant data or fail to produce coherent results, requiring careful human evaluation of quality and factual consistency [19]

7.4 Bias in Datasets

Multi-classification metrics depends on carefully curated benchmark datasets with perfectly labelled examples, which are costly and time-consuming to produce due to which LLMs struggle to handle ambiguous cases / multiple labels, and fail to reflect the complexity of real-world scenarios where labels are incomplete or unavailable [18].

7.5 Metric limitations

Question -answering metrics are limited in dialogue-based setting because they mainly focus on identifying answer boundaries rather than capturing deeper semantic understanding [18]. Similarly, ROUGE-based measure had difficulty in accounting for word variants and synonyms, making it challenging to accurately reflect the semantic quality of longer generated text [18].

8 Conclusion

This review synthesizes recent advancements in the fine-tuning of Transformer-based large language models, encompassing both small and large-scale implementations, and demonstrates that practical application now hinges more on sophisticated adaptation

methodologies than on mere parameter expansion. Case studies of TinyLlama, Gemma-2B, Phi-3-Mini, Llama-3-8B, and Mistral-7B illustrate that parameter -efficient techniques, including LoRA and QLoRA, alongside alignment strategies such as RLHF and DPO, can achieve constrained computational conditions. Concurrently ,persistent challenges such as hallucination, dataset bias, substantial hardware demands, and limited evaluation metrics highlight the necessity for more robust benchmark and more scalable alignment pipelines. The paper ultimately posits that future progress in LLM applications will depend on high-quality data curation, efficient fine-tuning frameworks, and more comprehensive evaluation methodologies, thereby indicating a shift.

9 References

[1] A. Vaswani *et al.*, "Attention Is All You Need," 2023.

[2] S. Minaee *et al.*, "Large Language Models: A Survey," Mar. 2025, [Online]. Available: http://arxiv.org/abs/2402.06196

[3] V. B. Parthasarathy, A. Zafar, A. Khan, and A. Shahid, "The ultimate guide to fine-tuning LLms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities," Oct.2024, [Online]. Available: http://arxiv.org/abs/2408.13296

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," May 2019, [Online]. Available: http://arxiv.org/abs/1810.04805

[5] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," Jan. 2020, [Online]. Available: http://arxiv.org/abs/2001.08361

[6] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," Mar. 2022, [Online]. Available: http://arxiv.org/abs/2203.02155

[7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners." [Online]. Available: https://github.com/codelucas/newspaper

[8] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," Feb. 2023, [Online]. Available: http://arxiv.org/abs/2302.13971

[9] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," Jul. 2020, [Online]. Available: http://arxiv.org/abs/2005.14165

[10] C. Raffel *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," Sep. 2023, [Online]. Available: http://arxiv.org/abs/1910.10683

[11] X. K. Wu *et al.*, "LLM Fine-Tuning: Concepts, Opportunities, and Challenges," *Big Data and Cognitive Computing*, vol. 9, no. 4, Apr. 2025, doi: 10.3390/bdcc9040087.

[12] "TinyLlama 1.1B-Size Doesn't Matter."

[13] Gemma Team *et al.*, “Gemma: Open Models Based on Gemini Research and Technology,” Apr. 2024, [Online]. Available: http://arxiv.org/abs/2403.08295

[14] M. Abdin *et al.*, “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone,” Aug. 2024, [Online]. Available: http://arxiv.org/abs/2404.14219

[15] S. Weerawardhena *et al.*, “Llama-3.1-FoundationAI-SecurityLLM-8B-Instruct Technical Report,” Aug. 2025, [Online]. Available: http://arxiv.corg/abs/2508.01059

[16] J. Walsh, S. Mamidanna, B. Nye, M. Core, and D. Auerbach, “Fine-tuning for Better Few Shot Prompting: An Empirical Comparison for Short Answer Grading,” 2025.

[17] Y. Moslem, R. Haque, and A. Way, “Fine-tuning Large Language Models for Adaptive Machine Translation,” Dec. 2023, [Online]. Available: http://arxiv.org/abs/2312.12740

[18] T. Hu and X.-H. Zhou, “Unveiling LLM Evaluation Focused on Metrics: Challenges and Solutions,” Apr. 2024, [Online]. Available: http://arxiv.org/abs/2404.09135

[19] P. Zhang, G. Zeng, T. Wang, and W. Lu, “TinyLlama: An Open-Source Small Language Model,” Jun. 2024, [Online]. Available: http://arxiv.org/abs/2401.02385